

BORDER OPTICS — POORI RESEARCH SAMJHO

PDF 9 of 22 — Acquisition: Cloud Masking

`extract_scl_cloud_mask_ndbi.py`, `scl_vs_qa60_comparison.py`
— Poori Line By Line

Sakshi ke liye, Sakshi ki apni research par. PDF 8 mein humne asli NDBI aur night-lights nikaali — QA60 method se cloud hataake. Us waqt humne ek chhota forward-pointer chhoda tha: 'ek behtar, refined cloud-masking method bhi hai, SCL, uski poori kahani PDF 9 mein.' Ye wahi PDF hai.

Ye do files, ek dusre ke saath, ek genuinely dramatic kahani sunati hain — ek cross-check jo shuru mein ek naya, exciting result deta hai, phir external review se ek chhupa hua bug pakda jaata hai (snow ko 'clear ground' maan liya gaya tha), aur fix hone ke baad wahi result palat jaata hai, wapas null ban jaata hai — is baar teen alag masks se confirm hoke. Isi ke saath-saath, comparison script mein khud ek doosra bug bhi milta hai: purane, hardcoded narrative-text jo naye data ke saath bhi purani kahani hi bolta reh gaya tha.

Shuru Karte Hain — Ye Do Files Kya Karti Hain, Aur Kyun

PDF 8 mein humne dekha ki `mask_s2_clouds()` function QA60 band use karta hai — Sentinel-2 ki apni built-in 'quality flag' — cloudy pixels hataane ke liye. Ye method is poore project ka original, primary method hai, aur is study ke saare headline results (H1, H4, sab) isi se aaye hain. Lekin koi bhi single method perfect nahi hota — har cloud-detection algorithm kuch galtiyan karta hai: kabhi genuine cloud ko miss kar deta hai, kabhi kisi aur cheez (jaise barf, ya bahut chamakdaar chhat) ko galti se 'cloud' maan leta hai. Isliye is research mein ek doosra, independent cross-check bhi banaya gaya — SCL (Scene Classification Layer) naam ka ek doosra Sentinel-2 band, jo QA60 se alag algorithm se, har pixel ko categorize karta hai.

EXAMPLE — Doosri Opinion Lena — Dusre Doctor Se Check Karwana

PDF 8 mein humne 'doctor ki checkup' analogy use ki thi — before/after vitals measure karna. Ab socho, wahi doctor apna measurement le chuka hai, lekin ek zaroori decision se pehle (jaise surgery), tu ek doosre doctor se bhi check karwati hai — second opinion. Agar dono doctors alag-alag measurement-technique use karke bhi same conclusion par pahunchte hain, tera bharosa badhta hai. Agar wo disagree karte hain, tu ruk jaati hai aur poochti hai 'kaun sahi hai, aur kyun.' SCL-mask cross-check exactly yahi role play karta hai — QA60 ka 'doosra doctor,' jo alag tareeke se cloud detect karke, wahi NDBI-change dobara measure karta hai.

Do files hain: pehli, `extract_scl_cloud_mask_ndbi.py` (folder `src/acquisition/` mein), jo SCL band use karke NDBI dobara nikaalti hai — do variants mein, 'corrected' aur 'strict.' Doosri, `scl_vs_qa60_comparison.py` — dhyaan se, ye file `src/acquisition/` mein nahi, balki `src/analysis/` folder mein rehti hai, kyunki ye naya data nahi nikaalti, sirf do already-extracted results ko compare karti hai. Ye ek chhoti lekin zaroori baat hai — file kaunsa kaam karti hai, uske hisaab se project use kis folder mein rakhta hai, sirf kaunse PDF mein padhai jaa rahi hai uske hisaab se nahi.

History Ek Nazar Mein — Ek Bug Ke Andar Ek Bug

Is PDF ki kahani teen stages mein hai, aur Development Log Entries 32 se 35 tak failti hai. Pehle stage mein (Entry 32), ye dono files likhi gayin, lekin turant nahi chalayi ja saki — kyunki inhe live Google Earth Engine access chahiye, jo sirf Sakshi ki apni machine par tha, is assistant ke paas nahi. Doosre stage mein (Entry 33), jab pehli baar chalayi gayin, ek chhupa hua bug — SCL class 11 (jo asal mein 'snow/ice' hai) ko 'clear ground' maan liya gaya — ek naya, exciting-lagne-wala result de deta hai: SCL-mask ke saath NDBI mein significant increase mil raha tha, jabki QA60 ke saath wahi period null tha. Teesre stage mein (Entry 34-35), external review se ye bug pakda jaata hai, fix hota hai do naye variants ke saath, aur — sabse zaroori — comparison

script mein khud ek doosra bug bhi milta hai, jahan print-hone-wala 'story' data se nahi, purane hardcoded text se aa raha tha. Ye poori PDF isi teen-stage safar ko, code ke exact un jagahon ke saath jahan har stage hua, dekhegi.

Part A — extract_scl_cloud_mask_ndbi.py: SCL Se Dobra NDBI Nikaalna

Docstring — Bug Ki Kahani Khud File Ke Top Par Likhi Hai

```
"""
SCL-mask cross-check for Section 6.10 -- redo summer NDBI (treated core sample
only) with the SCL band instead of QA60, see if the numbers hold up.

Bug caught in external review: class 11 is snow/ice, not clear ground, and
these are Himalayan villages in June-Sept, so the old class list was letting
snow through as "clear." Runs two corrected variants now instead of one --
"corrected" drops snow, "strict" also drops dark-area pixels -- so both are
on record instead of picking one. The old buggy output
(border_optics_village_results_summer_sclmask.csv, classes [2,4,5,6,11]) is
left on disk untouched as the pre-fix record, not overwritten.

Needs live GEE, run on my machine.

python3 src/acquisition/extract_scl_cloud_mask_ndbi.py
"""
```

Purpose: Ye docstring, doosri files se ek tarah se alag hai — ye sirf 'ye file kya karti hai' nahi batata, balki apni khud ki bug-history bhi record karta hai, seedha code ke andar. Pehla paragraph batata hai ye file kis liye hai: Section 6.10 ka cross-check, summer-window NDBI ko QA60 ki jagah SCL band se dobara nikaalna, treated core-sample villages ke liye. Doosra paragraph seedha bug confess karta hai: class 11 (Sentinel-2 ki SCL classification mein) 'snow/ice' hai, 'clear ground' nahi — aur kyunki ye poora sample Himalayan (high-altitude border) villages ka hai, June-September window mein bhi, purana class-list snow ko galti se 'saaf' maan raha tha. Fix ye hai ki do naye, corrected variants ban gaye — ek jo sirf snow hataata hai, doosra jo thoda zyada strict hai (dark-area pixels bhi hataata hai) — aur dono record par rakhe gaye, ek ko chunne ki jagah. Teesra paragraph zaroori hai: purana, buggy output file jaan-bujh kar disk par chhoda gaya hai, overwrite nahi kiya gaya — taaki pre-fix record bhi surakshit rahe, reproducibility ke liye. Aakhri lines batati hain ye file live Google Earth Engine access maangti hai, isliye sirf Sakshi ki apni machine par chalayi ja sakti hai. Syntax: Ek lamba, multi-paragraph triple-quoted docstring — module ke top par, usual jagah. Har paragraph ke beech ek khaali line, readability ke liye — Python isse ignore karta hai, ye sirf insaan padhne ke liye hai.

EXAMPLE — Entry 32 — Pehle 'Likhi Gayi,' Phir 'Chalayi Gayi'

Development Log Entry 32 mein ye dono files — `extract_scl_cloud_mask_ndbi.py` aur `scl_vs_qa60_comparison.py` — sabse pehle likhi gayin, jab research-completeness ka ek audit chal raha tha. Us waqt inhe sirf 'prepared, not run' ke roop mein record kiya gaya — kyunki live Earth Engine query chalane ke liye

Sakshi ki apni machine chahiye thi, is assistant ke paas nahi. Ye ek important pattern hai jo ab tak kai baar dikha hai (Entry 13 mein EE_PROJECT requirement, PDF 6-8 mein geocoding/extraction scripts) — jab bhi live external APIs (Earth Engine, Overpass, Nominatim) chahiye hoti hain, script yahan likhi jaati hai, documented hoti hai, lekin asal run Sakshi apni machine par karti hai, aur results wapas Development Log mein record hote hain.

Imports

```
import os
import time

import ee
import pandas as pd
from dotenv import load_dotenv

load_dotenv()
```

Purpose: Bilkul wahi imports jo PDF 8 ki dono files mein dekhe the — os environment-variable aur file-existence checks ke liye, time delay ke liye, ee Google Earth Engine ki library, pandas table-operations ke liye, aur dotenv se load_dotenv secret credentials (EE_PROJECT) .env file se load karne ke liye. Note karo — is file mein argparse import nahi hai, kyunki PDF 8 ki files ke ulat, is file mein command-line se koi option choose nahi karni — ye dono variants (corrected, strict) hamesha ek saath, dono chalaati hai, jaisa hum aage `__main__` block mein dekhenge. Syntax: Standard import statements, jaisa poore project mein consistent style. `load_dotenv()` file ke top par hi call hota hai, taaki neeche `os.environ.get(...)` se padhe jaane wale variables already load ho chuke hon.

Path Aur Buffer Constants

```
VILLAGES_PATH = "data/processed/border_optics_master_villages.csv"
BUFFER_RADIUS_M = 500
S2_COLLECTION = "COPERNICUS/S2_SR_HARMONIZED"
```

Purpose: Teenon constants bilkul wahi hain jo PDF 8 ki `extract_satellite_data.py` mein the — same master village-list (258 treated villages), same 500-meter buffer-radius, same Sentinel-2 collection. Ye jaan-bujh kar hai — cross-check ka poora point hi ye hai ki 'sab kuch same rakho, sirf cloud-masking method badlo,' taaki agar result alag aaye, tu confidently keh sako ki farak masking-choice ki wajah se hai, kisi aur cheez (jaise alag buffer-size ya alag village-set) ki wajah se nahi. Syntax: Teen module-level string/int constants — `VILLAGES_PATH` ek file-path string, `BUFFER_RADIUS_M` ek integer, `S2_COLLECTION` Earth Engine ka collection-identifier string.

BEFORE Aur AFTER — Summer Window, Hardcoded Is Baar

```
BEFORE = ("2021-06-01", "2021-10-01")
AFTER = ("2025-06-01", "2025-10-01")
```

Purpose: PDF 8 ki `WINDOWS` dictionary mein dono options — `full_year` aur `summer` — ek saath rakhe gaye the, kyunki wo primary extraction file thi, jahan dono windows genuinely chahiye the. Ye cross-check file sirf summer window ke liye hai — file ka apna docstring shuru mein hi 'redo summer NDBI' bolta hai — isliye yahan sirf ek pair of date-tuples, directly module-level par, hardcoded. Dates bilkul same hain jo PDF 8

ki summer window mein the: 2021 summer (before) vs 2025 summer (after), dono June 1 se October 1 tak. Syntax: Do tuples, har ek do date-strings ka — ("2021-06-01", "2021-10-01"). In variable-naamon (BEFORE, AFTER) ko poori file mein consistently use kiya jaayega, neechе ndbi_for_period_scl() ko call karte waqt.

VARIANTS — Jahan Snow-Bug Ki Poori Kahani Chhupi Hai

```
# corrected: 2 dark area, 4 vegetation, 5 bare soil, 6 water -- snow (11) dropped
# strict: same minus dark area pixels too, in case those are also iffy over villages
VARIANTS = {
    "corrected": {
        "classes": [2, 4, 5, 6],
        "out_path":
"data/processed/border_optics_village_results_summer_sclmask_corrected.csv",
    },
    "strict": {
        "classes": [4, 5, 6],
        "out_path":
"data/processed/border_optics_village_results_summer_sclmask_strict.csv",
    },
}
```

Purpose: Ye dictionary is poori file ka dil hai. Sentinel-2 ka SCL (Scene Classification Layer) band, har pixel ko ek number-code deta hai, jo batata hai wo pixel kis category ka hai: 2 = dark area (jaise shadow), 4 = vegetation, 5 = bare soil, 6 = water, aur — bahut zaroori — 11 = snow/ice. 'Corrected' variant sirf 2, 4, 5, 6 ko 'clear' maanta hai — snow (11) isse bahar hai. 'Strict' variant aur bhi kum classes rakhta hai — sirf 4, 5, 6 — dark-area pixels (2) ko bhi 'iffy' maan kar hata diya gaya, kyunki wo bhi kabhi-kabhi shadow ya partial-cloud jaisa dikh sakta hai. Dono variants apna alag out_path rakhte hain, taaki dono results record par rahein, ek dusre ko overwrite na karein. Syntax: Ek nested dictionary — bahar wali keys ("corrected", "strict") variant-naam hain, andar wali keys ("classes", "out_path") har variant ki settings. "classes" ki value ek Python list hai integers ki — [2, 4, 5, 6].

EXAMPLE — Class 11 — Woh Ek Number Jisne Poora Result Palat Diya

Ye VARIANTS dictionary jaise aaj dikh rahi hai, uske peeche teen Development Log entries ki poori kahani hai. Entry 32 mein, jab ye file pehli baar likhi gayi, class-list mein 11 (snow) bhi shaamil tha — sirf ek galti se, kisi ne assume kar liya ki snow bhi 'clear ground' hi ek prakaar hai, jabki asal mein wo bilkul alag surface-type hai. Entry 33 mein jab ye buggy version pehli baar live chalayi gayi, result exciting laga: SCL-mask ke saath, summer NDBI mein ek strong, significant increase mila (mean change +0.01169, $p=0.0000045$) — jabki QA60 method se wahi period hamesha null raha tha. Lekin Entry 34 mein, jab ek external review tool (iss project mein 'Anara' kehlaata hai) ne is exact class-list ko dhyaan se dekha, usne pakda: himalayan villages, June-September mein, snow ek bahut hi common aur significant surface hai — aur agar snow ko 'clear' maan liya jaaye, toh wahi snow-covered pixels galti se 'built-up jaisa' NDBI value de sakte hain, jo asal mein sirf ek masking-error hai, zameen par koi asli badlaav nahi. Isi discovery ne yahan dikh rahe do corrected variants ko janm diya — jo hum abhi dekh rahe hain.

init_ee() — Wahi Connection-Pattern, Ab Teesri Baar

```
def init_ee():
    project = os.environ.get("EE_PROJECT")
    try:
        ee.Initialize(project=project)
    except Exception:
        ee.Authenticate()
        ee.Initialize(project=project)
```

Purpose: Ye function, PDF 8 ki dono files ke `init_ee()` se, character-by-character identical hai — pehle environment se `EE_PROJECT` padhna, fir seedha `Initialize()` try karna, aur agar wo fail ho (jaise pehli baar `authenticate` na hui ho), `ee.Authenticate()` se browser-login flow chalaana aur dobara `Initialize()`. Ye function is project mein teesri baar dikh raha hai — `extract_satellite_data.py`, `extract_control_satellite_data.py`, aur ab yahan — aur teeno jagah wajah wahi hai: har script jo Earth Engine use karti hai, khud independently connect hoti hai, kyunki har script apne-aap chalti hai, alag Python process mein. Syntax: `os.environ.get("EE_PROJECT")` environment variable padhna, na milne par `None`. try/except Exception: broad catch, jaan-bujh kar — 'not authenticated' ki har possible error-type ka jawaab same hai. `ee.Authenticate()` browser flow shuru karta hai; uske baad `ee.Initialize(project=project)` dobara try hota hai.

make_mask_fn() — Ek Function Jo Function Banata Hai

```
def make_mask_fn(classes):
    def mask_s2_clouds_scl(image):
        scl = image.select("SCL")
        mask = scl.eq(classes[0])
        for c in classes[1:]:
            mask = mask.Or(scl.eq(c))
        return image.updateMask(mask).divide(10000)
    return mask_s2_clouds_scl
```

Purpose: Ye is file ka sabse naya, sabse interesting Python-pattern hai — ek 'function factory.' PDF 8 mein `mask_s2_clouds()` ek fixed, hardcoded function tha — hamesha wahi do QA60 bits check karta tha, kuch badal nahi sakta tha. Lekin yahan zaroorat alag hai: humein *do* alag masking-functions chahiye — ek jo classes [2,4,5,6] allow kare (corrected), ek jo [4,5,6] allow kare (strict) — aur dono ka core-logic same hona chahiye, sirf allowed-classes ki list alag. `make_mask_fn(classes)` ek 'outer' function hai jo *andar* ek 'inner' function `mask_s2_clouds_scl` define karta hai, aur us inner-function ko wapas bhejta hai — bina use call kiye, bas use wapas de deta hai, jaise ek blueprint. Jab baad mein `make_mask_fn([2,4,5,6])` call hoga, ye ek naya function wapas dega jo specifically classes 2,4,5,6 ko 'clear' maanega; jab `make_mask_fn([4,5,6])` call hoga, ek doosra, alag function milega. Andar, pehli class (`classes[0]`) se mask shuru hoti hai, fir baaki classes ke liye ek loop chalta hai jo har

naye class-check ko `.Or()` se pehle wale mask mein jodta jaata hai — matlab, 'agar pixel in classes mein se *koi bhi ek* hai, toh wo clear hai.' Syntax: Function-ke-andar-function — Python mein isse *closure* kehte hain. `make_mask_fn(classes)` ek naya function `def mask_s2_clouds_scl(image):` banata hai jiske andar wo apne bahar wale classes parameter ko 'yaad' rakhta hai, chahe outer-function ka kaam khatam ho chuka ho. `scl.eq(classes[0])` pehli class ke saath equality-check. `classes[1:]` list-slicing — list ke pehle element ke alawa baaki sab. `mask.Or(scl.eq(c))` — har iteration mein, purane mask ko naye class-check ke saath OR karna, taaki final mask un sab pixels ko True kare jo *kisi bhi* allowed class mein aate hain. `return mask_s2_clouds_scl` — function-object ko khud return karna, use call kiye bina (koi parentheses nahi `mask_s2_clouds_scl()` ke baad).

EXAMPLE — Do Function-Calls, Do Alag Behaviours — Bina Code Duplicate Kiye

Iska practical fayda samajhna zaroori hai. Agar closure use na karte, toh do bilkul alag functions likhne padte — ek jo `[2,4,5,6]` hardcoded rakhta, doosra jo `[4,5,6]` — aur dono ka baaki poora logic (mask-building, `.Or()` loop) copy-paste hota, jo maintenance ke liye risky hai (agar kal ek mein bug fix karna ho, doosre mein bhoolna asaan hai). Closure isse avoid karta hai — ek hi 'template' function hai, aur jab bhi ek naya variant chahiye, bas ek naya classes-list `make_mask_fn()` ko de do, wo apne-aap sahi function bana dega. Ye exact wahi software-design-reason hai jiske liye ye do-variant approach (corrected + strict, Entry 34) possible ho paayi — bina do baar poora extraction-script likhe.

ndbi_for_period_scl() — NDBI Nikaalna, Ab Ek mask_fn Parameter Ke Saath

```
def ndbi_for_period_scl(buffered_geom, start, end, mask_fn):
    collection = (
        ee.ImageCollection(S2_COLLECTION)
        .filterBounds(buffered_geom)
        .filterDate(start, end)
        .map(mask_fn)
    )
    count = collection.size().getInfo()
    if count == 0:
        return None, 0
```

Purpose: Ye function, PDF 8 ke `ndbi_for_period()` se bilkul milta-julta hai — same filtering-chain (`filterBounds`, `filterDate`, `.map(...)`), same zero-image null-safe return. Lekin ek key farak hai jo iska naam khud batata hai (`_scl` suffix): PDF 8 ka `ndbi_for_period()` hamesha wahi hardcoded `mask_s2_clouds` function use karta tha (`.map(mask_s2_clouds)`). Ye function ek chautha parameter leta hai, `mask_fn` — jo *upar wale `make_mask_fn()` se aaya hua, specific-variant-ke-liye-banaya-gaya closure* hai. Yahi wo jagah hai jahan closure-pattern ka fayda dikhta hai: yahi ek function, corrected variant ke liye bhi kaam karega aur strict variant ke liye bhi, sirf kaunsa `mask_fn` pass hota hai, uske hisaab se. **Syntax:** Chautha parameter `mask_fn` function-signature mein add hua hai. `.map(mask_fn)` — ab hardcoded function-naam ki jagah, parameter se aaya hua function pass ho raha hai. Baaki method-chain syntax bilkul same jo PDF 8 mein dekha.

```
composite = collection.median()
ndbi_image = composite.normalizedDifference(["B11", "B8"]).rename("NDBI")
stats = ndbi_image.reduceRegion(
    reducer=ee.Reducer.mean(), geometry=buffered_geom, scale=10, maxPixels=1e9,
).getInfo()
return stats.get("NDBI"), count
```

Purpose: Baaki poora function, PDF 8 ke `ndbi_for_period()` se exactly same hai — median composite banana, NDBI formula (B11 minus B8, divide by B11 plus B8 — SWIR1 aur NIR bands ka normalized difference), aur `reduceRegion` se buffer-area ka average nikaalna, 10-meter scale par. Ye jaan-bujh kar same hai — cross-check ka poora point yahi hai ki formula same rahe, sirf input-images ka cloud-masking alag ho. **Syntax:** `collection.median()`, `.normalizedDifference(["B11", "B8"])`, `.rename("NDBI")`, `reduceRegion(...)` — sab wahi syntax jo PDF 8 mein poori detail mein dekha gaya. `stats.get("NDBI")` safe dictionary-access, crash-proof agar value missing ho.

EXAMPLE — *BEFORE Aur *AFTER — Tuple Ko Do Arguments Mein 'Unpack' Karna

Aage, jab ye function call hogi (`extract()` ke andar), tu dekhegi `ndbi_for_period_scl(buffered_geom, *BEFORE, mask_fn)` jaisa kuch. Wo star (*) BEFORE se pehle ek zaroori Python-trick hai: BEFORE khud ek tuple hai, do dates ka

— ("2021-06-01", "2021-10-01"). Function ko do alag arguments chahiye — start aur end, alag-alag. *BEFORE likhne se, Python us tuple ko 'khol' (unpack) kar deta hai, aur uske dono elements ko do separate positional-arguments ki tarah pass kar deta hai — bina manually BEFORE[0], BEFORE[1] likhe. Ye chhota, lekin bahut common Python-pattern hai jab data pehle se hi tuple ki shakal mein stored ho.

extract() — Poore File Ka Orchestrator

```
def extract(classes, out_path, checkpoint_every=10):
    outcome_cols = [
        "ndbi_scl_before", "ndbi_scl_after",
        "ndbi_scl_before_image_count", "ndbi_scl_after_image_count",
    ]
```

Purpose: Ye function ek variant (corrected ya strict) ke liye poora extraction chalata hai. **Parameters:** `classes` (jaise `[2,4,5,6]`), `out_path` (kahan save karna hai), aur `checkpoint_every=10` (default — har 10 villages ke baad save). `outcome_cols`, PDF 8 ki files jaisa hi pattern hai, lekin sirf NDBI ke liye — is file mein night-lights nahi nikaale ja rahe, sirf NDBI, kyunki cross-check ka focus sirf NDBI par tha (jaisa docstring bhi kehta hai — VIIRS night-lights cloud-affected nahi hoti, isliye use SCL-recheck ki zaroorat nahi thi). **Syntax:** Function-definition with default parameter (`checkpoint_every=10`). List literal, chaar column-naam strings ka.

```
if os.path.exists(out_path):
    villages = pd.read_csv(out_path)
    print(f"Resuming from existing checkpoint: {out_path}")
else:
    villages = pd.read_csv(VILLAGES_PATH)
    villages = villages[villages["is_core_sample"] == True].reset_index(drop=True)
```

Purpose: Resume-logic ka pehla hissa — agar output-file pehle se exist karti hai (pichhli baar kahin ruk gayi thi), usi se resume karo. Warna, master village-list se shuru karo — lekin sirf `is_core_sample == True` wali rows, matlab sirf 251 core-sample villages, na ki saari 258 (jinme 7 Himachal illustrative villages bhi hain jo core statistical tests mein shaamil nahi hoti). Ye ek zaroori detail hai — PDF 8 ki primary extraction files saari 258 villages ke liye chalti thi, lekin ye cross-check sirf core-sample (251) ke liye hai, kyunki wahi actual H1 hypothesis-test mein use hoti hain. **Syntax:** `os.path.exists(path)` file-check. `villages[villages["is_core_sample"] == True]` boolean-mask filtering, jaisa PDF 7 mein dekha. `.reset_index(drop=True)` filtering ke baad dataframe ke row-index ko 0,1,2... se dobara number karta hai, purana index (jismein gaps ho sakte hain) ko drop karke.

```
for col in outcome_cols:
    if col not in villages.columns:
        villages[col] = None

mask_fn = make_mask_fn(classes)
```

Purpose: Missing columns ko None se bhar dena, PDF 8 jaisa hi safety-pattern. Fir, *yahi* wo jagah hai jahan closure asal mein banta hai — `make_mask_fn(classes)` ko is specific variant ke classes ke saath call karke, ek ready-to-use masking-function `mask_fn` mein store kiya jaata hai, jo aage loop ke har village ke liye reuse hoga. **Syntax:** Standard missing-column loop. `mask_fn = make_mask_fn(classes)` —

function-call ka result (jo khud ek function hai) ek variable mein store karna, taaki use baar-baar call kiya ja sake bina dobara make_mask_fn() chalayе.

Main Loop — Har Village Ke Liye SCL-Masked NDBI

```
for i, row in villages.iterrows():
    if all(pd.notna(row.get(c)) for c in outcome_cols):
        continue

    point = ee.Geometry.Point([row["longitude"], row["latitude"]])
    buffered_geom = point.buffer(BUFFER_RADIUS_M)

    ndbi_before, n_before = ndbi_for_period_scl(buffered_geom, *BEFORE, mask_fn)
    ndbi_after, n_after = ndbi_for_period_scl(buffered_geom, *AFTER, mask_fn)
```

Purpose: Har row ke liye — pehle skip-check (agar saare outcome columns already filled hain, ye village 'done' hai). Fir ek point-geometry banti hai, 500-meter buffer ke saath — bilkul PDF 8 jaisa. Fir `ndbi_for_period_scl()` ko do baar call kiya jaata hai — before window aur after window ke liye — dono baar wahi `mask_fn` pass karte hue, jo upar is variant ke liye banaya gaya tha. Yahan `*BEFORE` aur `*AFTER` wahi tuple-unpacking hai jo pichhle EXAMPLE mein explain hui. Syntax: `all(pd.notna(row.get(c)) for c in outcome_cols)` — generator-expression, jaisa PDF 8 mein dekha. `ee.Geometry.Point([lon, lat])`, `.buffer(...)` — same Earth Engine geometry-syntax. Function-call mein `*BEFORE`, `*AFTER`, aur `mask_fn` — teen arguments (do unpacked, ek direct).

```
villages.at[i, "ndbi_scl_before"] = ndbi_before
villages.at[i, "ndbi_scl_after"] = ndbi_after
villages.at[i, "ndbi_scl_before_image_count"] = n_before
villages.at[i, "ndbi_scl_after_image_count"] = n_after

if (i + 1) % checkpoint_every == 0:
    villages.to_csv(out_path, index=False)
    print(f" {i + 1}/{len(villages)} villages processed...")

time.sleep(0.2)
```

Purpose: Results dataframe ki row mein likhe jaate hain — column-naam is baar `ndbi_scl_...` prefix ke saath, taaki agar kabhi dono (QA60 aur SCL) results ek hi file mein ho, confuse na ho. Fir checkpoint-save (har 10 villages), aur ek chhota 0.2-second wait — wahi Earth-Engine-overload-na-karne wala pattern. Syntax: `Chaar .at[i, "col"] = value` single-cell assignments. Modulo-based checkpoint-check, jaisa poore project mein consistent hai. `time.sleep(0.2)`.

```
villages.to_csv(out_path, index=False)
n_valid = villages.dropna(subset=["ndbi_scl_before", "ndbi_scl_after"]).shape[0]
print(f"Done. {n_valid}/{len(villages)} villages have valid SCL-masked before/after NDBI.")
print(f"Saved to {out_path}")
```

Purpose: Loop khatam hone ke baad, final save aur summary — kitni villages ke paas genuinely valid (dono before aur after) SCL-masked NDBI hai. Ye n_valid number hi aage decide karega ki 'coverage' kitni hai — Entry 33 mein ye 200/258 tha (buggy version), Entry 34 mein fix hone ke baad 251/251 (matlab, saari core-sample villages valid nikli). Syntax: `.dropna(subset=[col1, col2]), .shape[0]` — usual row-count pattern.

__main__ Block — Dono Variants Ek Hi Run Mein

```
if __name__ == "__main__":
    init_ee()
    for name, cfg in VARIANTS.items():
        print(f"\n=== variant: {name} ===")
        extract(cfg["classes"], cfg["out_path"])
```

Purpose: Jab file seedhe chalayai jaati hai, sabse pehle Earth Engine se connect hota hai — ek hi baar, dono variants ke liye reuse hoga. Fir VARIANTS dictionary ke har entry (corrected, phir strict) ke liye, ek header print hota hai, aur extract() call hota hai us variant ke classes aur out_path ke saath. Ye PDF 8 ki files se alag hai — wahan --window argparse-flag se ek run mein ek hi window chalti thi; yahan koi command-line choice nahi hai, dono variants hamesha ek saath, ek hi run mein chalte hain, kyunki dono record par rakhne ka poora point hi Entry 34 ka decision tha. **Syntax:** for name, cfg in VARIANTS.items(): — dictionary ke saare key-value pairs par loop, .items() se dono ek saath milte hain. f"\n=== variant: {name} ===" — f-string ke andar \n ek extra khaali line print karta hai, readability ke liye. extract(cfg["classes"], cfg["out_path"]) — nested-dictionary se do values nikaal kar function ko positional arguments ki tarah pass karna.

EXAMPLE — Entry 34 — Dono Variants Clean Chale

Jab ye fixed version pehli baar chali (Entry 34), dono variants ne 251/251 villages ke liye clean, bina kisi error ke, valid data diya — pura core sample cover ho gaya, missing kuch nahi. Ye khud ek chhota lekin zaroori signal hai: bug fix hone ke baad, extraction process khud stable aur reliable nikla, koi naya masking-crash nahi aaya.

Part B — scl_vs_qa60_comparison.py: Dono Masks Ko Aamne-Saamne Rakhna

Ab doosri file. Iska docstring saaf batata hai: *'Compares the QA60-masked primary summer NDBI result against the SCL-masked re-extraction... Run this after that extraction script has completed.'* Ye file, Part A ki file ka 'natural partner' hai — Part A naya data nikaalti hai, ye file us naye data ko purane (QA60) data ke saath compare karti hai, aur ek statistical verdict deti hai: kya dono masks agree karte hain? Yaad rakhna — ye file `src/acquisition/` mein nahi, `src/analysis/` mein rehti hai, kyunki ye analysis hai, acquisition nahi.

Docstring Aur Path Constants

```
"""Compares the QA60-masked primary summer NDBI result against the
SCL-masked re-extraction from extract_scl_cloud_mask_ndbi.py. Run this after
that extraction script has completed.
"""
```

```
import json
```

```
import pandas as pd
from scipy import stats
```

Purpose: Docstring short hai, seedha batata hai kaam kya hai aur ise kab chalana hai (Part A ke extraction ke *baad*). Imports mein json naya hai — is file ka final output ek JSON file hai, na ki CSV (kyunki ye raw data nahi, results-ka-summary save kar rahi hai). pandas usual table-kaam ke liye. from scipy import stats — naya, is file ka sabse important import — SciPy library ka stats module, jisme statistical tests hote hain (Wilcoxon, Spearman, jo hum aage dekhenge). Note karo — koi ee import nahi hai yahan, kyunki ye file Earth Engine se kuch nahi maangti, sirf pehle se-nikaale-hue CSV files padhti hai. Syntax: Triple-quoted docstring. import json, import pandas as pd — standard. from scipy import stats — module ke andar se ek specific sub-module import karna, taaki aage stats.wilcoxon(...) jaisa likha ja sake, scipy.stats.wilcoxon(...) ki jagah.

```
QA60_PATH = "data/processed/border_optics_village_results_summer_analyzed.csv"
# corrected = snow (class 11) dropped -- the old sclmask.csv file (no "_corrected"
# suffix) still exists on disk but was extracted with the buggy class list, don't use it
SCL_PATH =
"data/processed/border_optics_village_results_summer_sclmask_corrected.csv"
OUT_JSON = "outputs/scl_vs_qa60_comparison.json"

ALPHA = 0.05
```

Purpose: QA60_PATH PDF 8 ki primary summer-extraction ka, analysis-ke-baad-wala output hai (`_analyzed` suffix — pehle se hi `ndbi_change` column ke saath). SCL_PATH Part A ke 'corrected' variant ka output hai — aur is line ke upar wala comment bahut zaroori hai: ye seedha, code ke andar, ek warning likh kar rakhta hai ki purani buggy file (`..._sclmask.csv`, bina suffix ke) abhi bhi disk par maujood hai, lekin use mat karo, kyunki wo buggy class-list se banayi gayi thi. Ye exact wahi discipline hai jo docstring mein bhi thi — purani file ko delete nahi kiya gaya (record ke liye), lekin code explicitly bata deta hai ki wo 'use mat karo.' OUT_JSON final comparison-result kahan save hoga. ALPHA = 0.05 — significance-threshold, standard 5% level, jo poore project mein consistent hai. Syntax: Char module-level constants. Comment do lines mein span karta hai, readability ke liye. ALPHA = 0.05 ek float constant, statistical tests mein compare karne ke liye reuse hoga.

EXAMPLE — Ek Comment Jo Khud Ek Safety-Check Hai

Ye SCL_PATH ke upar wala comment, coding mein ek bahut achhi practice ka example hai. Sirf bug fix karna kaafi nahi hota — agar purani, galat file wahin padi rahe aur naye code ke paas usse bachne ka koi clear reason na ho, koi bhi (khud tu, mahino baad) galti se purani file phir se use kar sakta hai. Ye comment is risk ko seedha address karta hai — kisi ko bhi is variable ko badalne se pehle, saamne hi warning dikh jaati hai. Development Log Entry 34 mein bhi yahi discipline dikhi thi Part A ki file mein, jahan purani buggy CSV ko overwrite nahi kiya gaya.

main() — Data Load Karna Aur Merge Karna

```
def main():
    qa60 = pd.read_csv(QA60_PATH)
    qa60 = qa60[qa60["is_core_sample"] == True]
    scl = pd.read_csv(SCL_PATH)
    scl["ndbi_scl_change"] = scl["ndbi_scl_after"] - scl["ndbi_scl_before"]
```

Purpose: Dono CSVs padhi jaati hain — QA60 ka result, aur SCL ka result. QA60-side mein, sirf core-sample rows rakhi jaati hain (bilkul jaisa Part A ki extract() mein kiya tha, isliye consistent hai). SCL-side mein ek naya column banaya jaata hai turant — ndbi_scl_change, jo after-minus-before hai. Interesting baat: QA60 file mein ye 'change' column pehle se hi tha (_analyzed suffix isi liye — matlab, wo file already ek analysis-step se guzar chuki hai), lekin SCL file mein ye column nahi tha (Part A sirf raw before/after values save karti hai), isliye ye line yahan khud calculate karti hai. **Syntax:** pd.read_csv(path) — dono baar. Boolean-mask filtering (qa60[qa60["is_core_sample"] == True]). Naya column banana ek simple subtraction se — scl["col"] = scl["colA"] - scl["colB"] — pandas mein ye poore column par ek saath (vectorized) hota hai, row-by-row loop ki zaroorat nahi.

```
merged = qa60[["village_id", "ndbi_change"]].merge(
    scl[["village_id", "ndbi_scl_before_image_count", "ndbi_scl_after_image_count",
        "ndbi_scl_change"]],
    on="village_id", how="inner",
)

valid = merged.dropna(subset=["ndbi_change", "ndbi_scl_change"])
```

Purpose: Dono dataframes ko village_id ke through jod kar, ek naya merged dataframe banaya jaata hai — jisme har village ke liye, QA60 ka NDBI-change bhi hai aur SCL ka NDBI-change bhi, saath-saath. how="inner" ka matlab hai — sirf wo villages rakho jo dono files mein maujood hain; agar koi village sirf ek mein hai, wo merge se bahar reh jaayegi. Fir valid banaya jaata hai — un dono change-columns mein se jahan bhi koi missing (NaN) ho, wo row hata di jaati hai, taaki aage ka village-level comparison sirf genuinely-complete data par ho. **Syntax:** qa60[["col1", "col2"]] — double square-brackets se, ek chhota dataframe banaya jaata hai sirf specific columns ka, merge se pehle. .merge(other_df, on="village_id", how="inner") — pandas ka SQL-jaisa join; how="inner" sirf dono taraf matching rows rakhta hai (PDF 8 mein how="left" dekha tha, jo saari rows rakhta; yahan inner jaan-bujh kar hai kyunki agreement-analysis ke liye sirf wahi villages chahiye jo dono jagah maujood hon). .dropna(subset=[...]) — usual missing-value removal.

Teen Alag Wilcoxon Tests — Kyun Ek Nahi, Teen

```
scl_valid = scl.dropna(subset=["ndbi_scl_before", "ndbi_scl_after"])
w_stat_scl, w_p_scl = stats.wilcoxon(
    scl_valid["ndbi_scl_after"], scl_valid["ndbi_scl_before"], alternative="greater"
)
```

Purpose: Ye pehla test hai — SCL-masked data par, khud apne-aap. `scl_valid` wo villages hain jinke paas dono (before aur after) SCL-masked NDBI hai. `stats.wilcoxon(after_values, before_values, alternative="greater")` ek *Wilcoxon signed-rank test* chalata hai — ye ek non-parametric statistical test hai jo check karta hai ki 'after' values, 'before' values se significantly zyada hain ya nahi, paired samples ke through (matlab, har village ka apna before-after pair compare hota hai, na ki do alag groups). `alternative="greater"` ka matlab hai — hum specifically test kar rahe hain 'kya after zyada hai before se,' na ki sirf 'kya farak hai' (jo "two-sided" hota). Syntax: `stats.dropna(...)`-jaisa `dropna` call, familiar pattern. `stats.wilcoxon(x, y, alternative="greater")` — SciPy function, do array-jaise inputs leta hai, ek tuple return karta hai (test-statistic, p-value), jo tuple-unpacking se `w_stat_scl`, `w_p_scl` mein aata hai.

```
rho, p_corr = stats.spearmanr(valid["ndbi_change"], valid["ndbi_scl_change"])
n_qa60_valid = qa60["ndbi_change"].notna().sum()
n_scl_valid = len(scl_valid)
```

Purpose: Ye ek doosri tarah ka test hai — *Spearman correlation*. Wilcoxon test ye batata hai 'kya overall trend hai' (average-jaisa), lekin Spearman ye batata hai 'kya dono masks village-by-village agree karte hain' — matlab, agar ek gaon mein QA60 ke hisaab se NDBI badha, kya usi gaon mein SCL ke hisaab se bhi badha? Rho (rho) ek number hai -1 se +1 ke beech — jitna 1 ke paas, utna zyada agreement. `n_qa60_valid` aur `n_scl_valid` sirf dono masks ki 'coverage' (kitni villages ke paas valid data hai) record karte hain, aage print/compare karne ke liye. Syntax: `stats.spearmanr(x, y)` SciPy function, do arrays leta hai, tuple (rho, p_value) deta hai. `.notna().sum()` — `notna()` ek True/False Series deta hai (missing nahi hai toh True), `.sum()` un True values ko ginta hai (Python mein True=1 maana jaata hai jab sum kiya jaaye). `len(scl_valid)` dataframe ki row-count.

```

common_ids = set(scl_valid["village_id"])
qa60_matched =
qa60[qa60["village_id"].isin(common_ids)].dropna(subset=["ndbi_change"])
w_stat_qa60_matched, w_p_qa60_matched = stats.wilcoxon(
    qa60_matched["ndbi_after"], qa60_matched["ndbi_before"], alternative="greater"
)

```

Purpose: Ye is file ka sabse zaroori design-decision hai, aur code ke comment mein khud reasoning likhi hai (thodi neeche dikhegi): QA60 ko sirf *unhi* villages tak restrict karna jo SCL-mask mein bhi valid hain — na ki QA60 ke apne poore 251-village sample par. Kyun? Kyunki agar SCL aur QA60 ki 'coverage' (kitni villages valid hain) alag-alag ho, toh agar tu unke results seedhe compare kare, tu pakka nahi bol sakti ki farak *masking-method* ki wajah se hai ya *sample-composition* ki wajah se (jaise agar SCL sirf ek particular state ki villages ko zyada 'valid' maan raha ho). Isi confound ko avoid karne ke liye, ye 'matched subsample' banaya jaata hai — QA60 ka data, lekin sirf unhi village_ids ke liye jo SCL mein bhi valid hain — aur usi matched-subsample par QA60 ka apna Wilcoxon test chalaya jaata hai. Syntax: `set(scl_valid["village_id"])` — ek Python set banana ek column ki values se, fast membership-check ke liye. `qa60["village_id"].isin(common_ids)` — pandas ka `.isin()`, har row check karta hai ki uska `village_id` us set mein hai ya nahi, True/False Series deta hai. `qa60[boolean_series].dropna(...)` — filter aur dropna, chained ek saath. Wilcoxon test wahi pattern, is baar matched-subsample par.

EXAMPLE — Entry 33 Ka Vaada — Sample-Composition Ko Blame Na Karo Bina Check Kiye

Development Log Entry 33 mein, jab pehli baar (buggy) SCL result QA60 se alag aaya tha, ek zaroori question uthaya gaya: kya ye farak sirf isliye hai kyunki SCL-valid 200 villages, sample composition mein alag thi (us waqt woh 93% Arunachal Pradesh thi)? Us waqt, isi qa60_matched-jaisi approach se check kiya gaya — QA60 ko usi 200-village sample tak restrict karke — aur pata chala ki QA60 fir bhi null hi raha ($p=0.449$), matlab sample-composition akela is farak ko explain nahi kar raha tha, mask-choice khud kuch kar raha tha. Ye exact code-pattern us discipline ko har future comparison ke liye permanently built-in kar deta hai — sirf ek baar manually check karne ki jagah, ab ye har run mein automatically hota hai.

```
w_stat_qa60_full, w_p_qa60_full = stats.wilcoxon(
    qa60.dropna(subset=["ndbi_before", "ndbi_after"])["ndbi_after"],
    qa60.dropna(subset=["ndbi_before", "ndbi_after"])["ndbi_before"],
    alternative="greater"
)
```

Purpose: Teesra aur aakhri Wilcoxon test — QA60 ke apne *poore* sample par (sirf matched-subsample nahi, balki jitni bhi villages QA60 mein khud valid hain). Ye PDF 8/11 mein already-established 'primary' result hai — yahan sirf reference ke liye dobara compute kiya ja raha hai, taaki teenon numbers (SCL, QA60-matched, QA60-full) ek hi jagah, ek hi script se consistently print ho sakein, alag-alag jagah se copy-paste karke numbers mismatch hone ka risk na ho. Syntax: Wahi stats.wilcoxon(...) pattern, teesri baar. Is baar dropna do baar call ho raha hai (ek before-column ke liye, ek after ke liye, dono baar same subset) — thoda repetitive, lekin functionally sahi, kyunki .dropna(subset=[col1, col2]) already dono columns check kar raha hai ek hi call mein.

qa60_sig, scl_sig, agree — Jahan Dynamic-Narrative Bug Tha

```
qa60_sig = w_p_qa60_matched < ALPHA
scl_sig = w_p_scl < ALPHA
agree = qa60_sig == scl_sig
```

Purpose: Ye teen lines chhoti dikhti hain, lekin Development Log Entry 34 ki sabse zaroori fix yahin hai. qa60_sig aur scl_sig — dono simple boolean values hain, 'kya ye p-value 0.05 se kam hai?' agree — 'kya dono ek jaisa verdict de rahe hain (dono significant, ya dono not-significant)?' Ye teeno values *har baar jab script chalti hai, actual data se compute hoti hain* — koi bhi hardcoded True/False nahi. Ye design choice hi wo cheez hai jisne Entry 34 ke bug ko fix kiya. Syntax: w_p_qa60_matched < ALPHA — comparison operator, result ek boolean (True/False). qa60_sig == scl_sig — equality-check do booleans ke beech; Python mein True == True ya False == False dono True dete hain (matlab 'agree'), True == False False deta hai (matlab 'disagree').

EXAMPLE — Entry 34 Ka Doosra Bug — Jab Script Ka Print Khud Jhooth Bol Raha Tha

Jab pehli baar (buggy, Entry 33) ye comparison chalaya gaya tha, QA60 aur SCL disagree kar rahe the — QA60 null, SCL significant. Us waqt is script ke print-statements ne ek specific, hardcoded sentence likha: 'the mask choice ALONE flips the result... 93% Arunachal Pradesh...' — ye sentence seedha text ke roop mein likhi gayi thi, data se compute nahi ho rahi thi. Jab Entry 34 mein Part A ka snow-bug fix hua aur SCL ka result bhi null ho gaya (matlab ab dono agree karte hain), ye purana hardcoded sentence, jo kabhi update hi nahi hui thi, *abhi bhi wahi purani, ab-galat kahani print kar rahi thi* — ki dono disagree karte hain, jabki asal data mein ab dono agree karte the. Ye bug sirf isliye pakda gaya kyunki kisi ne actual script ko dhyaan se padha, sirf uske output par bharosa nahi kiya. Fix yahi hai jo tu abhi dekh

rahi hai — agree ab hamesha live-compute hota hai, qa60_sig aur scl_sig se, isliye ab ye galat/purana verdict print karna *structurally hi possible nahi* — jab tak actual numbers na badlein, verdict bhi nahi badlega, aur agar numbers badlein, verdict apne-aap sahi update ho jaayega.

Print Statements — Ab Sab Kuch Data Se Aata Hai

```
print("=== QA60 mask vs SCL mask, summer-window NDBI, treated core sample ===\n")
print(f"QA60-masked (primary, full n={n_qa60_valid}):
mean_change={qa60['ndbi_change'].mean():+.5f}, "
      f"p={w_p_qa60_full:.6f} ({'significant' if w_p_qa60_full < ALPHA else 'not
significant'})")
print(f"SCL-masked (this check, n={n_scl_valid}):
mean_change={scl_valid['ndbi_scl_change'].mean():+.5f}, "
      f"Wilcoxon p={w_p_scl:.6f} ({'significant' if scl_sig else 'not significant'})")
```

Purpose: Ye teen print-statements headline-numbers dikhate hain — pehle ek section-title, fir QA60 ka full-sample result, fir SCL ka result. Dhyaan do — har 'significant'/'not significant' label bhi *inline* compute ho raha hai, seedha us print-statement ke andar, ek conditional-expression se — koi bhi fixed text-string kahin nahi hai. Ye bilkul wahi discipline hai jo Entry 34 ne enforce ki — har cheez jo 'verdict' jaisi lagti hai, actual numbers se derive honi chahiye, us hi jagah jahan print ho rahi hai. Syntax: `+.5f` aur `:.6f` — f-string format-specifiers; `+.5f` ka matlab hai 5 decimal places, aur number ke aage hamesha + ya - sign dikhao (chahe positive ho ya negative), `.6f` 6 decimal places, bina forced sign ke. `{'significant' if condition else 'not significant'}` — ek inline conditional-expression (ternary), f-string ke andar hi — 'agar condition True hai toh ye text, warna wo text,' bina alag if/else block likhe.

```
print(f"\nSame-sample check (QA60 restricted to the identical {len(qa60_matched)}
SCL-valid villages): "
      f"mean_change={qa60_matched['ndbi_change'].mean():+.5f}, Wilcoxon
p={w_p_qa60_matched:.6f} "
      f"({'significant' if qa60_sig else 'not significant'})")

if agree:
    print(f"--> Holding the village sample fixed, the two masks AGREE: both "
          f"({'significant' if qa60_sig else 'null'}.)")
else:
    print(f"--> Holding the village sample fixed, the mask choice ALONE flips the result: "
          f"{'QA60 significant, SCL null' if qa60_sig else 'QA60 null, SCL significant'}.")
```

Purpose: Pehla print, matched-subsample ka result dikhata hai. Fir ek if agree: ... else: ... block — *yahi* wo exact jagah hai jahan purana hardcoded bug tha, aur ab jo fixed version hai. Agar agree True hai (jo Entry 34 ke baad ki actual situation hai), ek dynamic sentence print hoti hai jo khud bataati hai ki dono 'significant' hain ya 'null' — qa60_sig se decide hokar, hardcoded nahi. Agar agree False hota (jaisa buggy Entry 33 mein tha), doosra branch chalta, jo bhi batata ki specifically kaun significant hai aur kaun null — wo bhi qa60_sig se compute hokar, na ki fixed '93% Arunachal' jaisi text se. Syntax: if agree: ... else: ... — standard Python conditional, do poori branches ke saath, dono print-statements. Andar dono jagah conditional-expressions ('X' if cond else 'Y') hain, jo runtime par actual boolean values dekh kar decide karte hain kaunsa

text dikhana hai — is poore if/else ka structure khud static hai (hamesha same do branches), lekin har branch ke andar ka content dynamic hai.

Village-Level Agreement Aur Coverage — Jahan Entry 35 Ka Bug Tha

```
print(f"\nVillage-level agreement between the two masks' NDBI change (n={len(valid)})
villages valid at both): "
    f"Spearman rho={rho:+.4f}, p={p_corr:.4g}")

coverage_diff = n_scl_valid - n_qa60_valid
print(f"\nCoverage: SCL mask returns valid data for {n_scl_valid}/251 core-sample
villages vs "
    f"QA60's {n_qa60_valid}/251 ({'+' if coverage_diff >= 0 else ''}{coverage_diff}). "
    f"If this gap is large, check state composition separately -- don't assume it's the "
    f"same skew an earlier run had; that's exactly the kind of assumption that goes
stale.")
```

Purpose: Pehla print, Spearman village-level agreement dikhata hai — kitni villages ke paas dono masks mein valid data hai (`len(valid)`), aur rho/p-value. Fir `coverage_diff` — SCL ki coverage minus QA60 ki coverage, ek simple subtraction. Aakhri print mein, dono coverage-numbers ke saath `/251` hardcoded hai — 251 core-sample ki total count, jo yahan seedha likh diya gaya hai bina variable ke, kyunki ye fix hi hai (core-sample ka size badalne ka koi tareeka is script mein nahi hai). Sabse important, aakhri sentence: ye explicitly ek future-warning likhta hai — 'agar coverage-gap bada dikhe, sample-composition alag se check karo, purani run ka wahi skew mat maan lo.' Ye seedha Entry 33 ke experience se seekha gaya lesson hai, code ke comment ki jagah, ek printed-warning ke roop mein baked-in kar diya gaya. Syntax: `coverage_diff = n_scl_valid - n_qa60_valid` — simple integer subtraction. '+' if `coverage_diff >= 0` else '' — conditional-expression jo positive numbers ke aage explicit + sign lagata hai (Python integers negative number ke liye khud - laga deta hai, lekin positive ke liye kuch nahi, isliye ye manually handle kiya gaya hai).

EXAMPLE — 200 Se 251 Tak — Ek Sawaal Jo Khula Chhoda Gaya, Chhupaya Nahi Gaya

Entry 34 mein ek genuinely ajeeb cheez dekhi gayi: buggy version (Entry 33) mein SCL-valid coverage 200/258 thi; fix hone ke baad (Entry 34), coverage badh kar 251/258 ho gayi. Ye mechanically thoda ajeeb hai — jab tu ek class ko 'clear' list se *hataati* hai (yahan snow, class 11), coverage sirf same reh sakti hai ya kam ho sakti hai, badhni nahi chahiye, kyunki ab pehle se *kam* pixel-types 'clear' maane ja rahe hain. Is increase ko poori honesty se explain nahi kiya gaya — sabse zyada likely wajah, Sentinel-2 ke satellite-archive ka wahi 'backfilling' phenomenon hai jo Entries 21-22 mein bhi dekha gaya tha (jahan Earth Engine, do extraction-runs ke beech, purane scenes ko archive mein add kar deta hai, isliye same query, alag dates par chalane se, alag image-counts de sakti hai) — lekin ye directly confirm nahi kiya gaya, sirf ek plausible explanation ke roop mein flag kiya gaya hai. Ye is poore project ki ek consistent achhi aadat dikhata hai — jab kuch fully explain nahi hota, use chhupaya nahi jaata ya guess ko fact ki tarah pesh nahi kiya jaata; use khule taur par 'open question' maan kar record kiya jaata hai.

EXAMPLE — Entry 35 — JSON File Khud Purani Reh Gayi Thi

Ek teesra, alag bug bhi mila, thoda baad mein (Entry 35), ek aur external review ke dauraan. Script khud, is poore PDF mein jo dekha, sahi tarike se fix ho chuki thi — lekin jo actual outputs/scl_vs_qa60_comparison.json file Sakshi ki machine par deliver hui thi, uske andar coverage_note (ek text-field jo JSON mein save hota hai) abhi bhi purani, pre-fix kahani keh raha tha — chahe baaki saare numeric fields already sahi the. Root cause: fix hone ke baad, JSON sandbox mein regenerate karke verify toh kiya gaya tha, lekin delivery-batch mein sirf fixed *script file* hi device par bheji gayi thi — naya, sahi JSON file khud sync nahi hui thi. Fix simple tha — dobara extraction ki zaroorat nahi thi, bas already-sahi JSON ko phir se sync karna tha. Ye ek achha reminder hai ki 'code fix ho gaya' aur 'us code ka har output wahan pahuncha jahan chahiye tha,' ye do alag cheezein hain — dono ko alag se verify karna padta hai.

result Dictionary Aur JSON Save

```
result = {
    "n_qa60_valid": int(n_qa60_valid), "qa60_mean_change":
float(qa60["ndbi_change"].mean()),
    "qa60_wilcoxon_p_full": float(w_p_qa60_full),
    "n_scl_valid": int(n_scl_valid), "scl_mean_change":
float(scl_valid["ndbi_scl_change"].mean()),
    "scl_wilcoxon_p": float(w_p_scl),
    "qa60_matched_subsample_n": int(len(qa60_matched)),
    "qa60_matched_subsample_mean_change":
float(qa60_matched["ndbi_change"].mean()),
    "qa60_matched_subsample_wilcoxon_p": float(w_p_qa60_matched),
    "village_level_agreement_rho": float(rho), "village_level_agreement_p": float(p_corr),
    "n_common_valid": int(len(valid)),
    "masks_agree_on_significance": bool(agree),
    "coverage_diff": int(coverage_diff),
}
```

Purpose: Ye dictionary is poore comparison ka final, saved summary hai — har number jo print hua, yahan bhi hai, plus masks_agree_on_significance (wahi agree boolean, JSON mein save hone layak naam ke saath) aur coverage_diff. Dhyaan do — purani buggy version mein ek coverage_note naam ka text-field bhi tha (jo Entry 35 mein stale reh gaya tha); is current, fixed version mein wo text-field hi hata diya gaya hai — sirf numeric aur boolean fields bache hain, jo khud data se seedhe compute hote hain. Ye khud fix ka hissa hai: agar koi free-text 'narrative' field JSON mein nahi hai, toh wo kabhi bhi 'stale' nahi ho sakta, kyunki wo hai hi nahi. **Syntax:** Dictionary literal, har value explicitly type-cast ki gayi hai — int(...), float(...), bool(...). Ye zaroori hai kyunki pandas/numpy apne khud ke number-types use karte hain (jaise numpy.float64, numpy.bool_), jo Python ki built-in json library seedhe samajh nahi paati — cast karne se wo plain Python int/float/bool ban jaate hain, jo JSON mein safely likhe ja sakte hain.

```
with open(OUT_JSON, "w") as f:
    json.dump(result, f, indent=2)
    print(f"\nSaved {OUT_JSON}")

if __name__ == "__main__":
    main()
```

Purpose: File ko write-mode mein khola jaata hai, aur json.dump() se poora dictionary, formatted JSON ki tarah, us file mein likh diya jaata hai. Fir ek confirmation-print. Aakhir mein, __main__ block bas main() ko call karta hai — is file mein koi command-line arguments nahi hain, kyunki iske paas koi choice hi nahi hai kya karna hai, sirf ek hi, fixed comparison chalana hai. **Syntax:** with open(path, "w") as f: — file-handling ka standard, safe pattern (with automatically file close kar deta hai, chahe

error aaye ya na aaye). `json.dump(obj, file, indent=2)` — dictionary ko JSON text mein badal kar file mein likhta hai; `indent=2` output ko human-readable banata hai (nested, indented structure ke saath), bajaye sab kuch ek hi line mein likhne ke.

Synthesis — Teen Masks, Ek Hi Jawaab

Ab poori kahani ko ek jagah jodte hain. Summer-window treated core-sample NDBI change ka sawaal — 'kya border villages mein satellite se dikhne wala built-up area significantly badha' — teen alag, independent cloud-masking approaches se test kiya gaya. Pehle QA60 (primary method, jo poore study ka backbone hai), fir SCL-corrected (snow-bug fix ke baad), fir SCL-strict (dark-area bhi hata kar, ek aur zyada conservative check). Teenon ka verdict same hai: null.

Mask / Variant	n (valid villages)	Mean NDBI Change	Wilcoxon p-value	Significant?
QA60 (primary)	251	-0.00090	0.43583	Nahi
SCL — corrected [2,4,5,6]	251	+0.00112	0.28797	Nahi
SCL — strict [4,5,6]	251	+0.00187	0.17501	Nahi

Village-level agreement bhi, bug fix ke baad, kaafi mazboot ho gaya — QA60 aur SCL-corrected ke beech Spearman rho 0.734 (pehle, buggy version mein sirf 0.544 tha), aur QA60 aur SCL-strict ke beech rho 0.736. Matlab, sirf ye nahi ki teeno ka average verdict same hai — individual villages ke level par bhi, teeno masks ab kaafi hadd tak ek-dusre se sehmat hain, jahan pehle (buggy version mein) sirf moderate agreement (0.544) tha.

Ye Ek Achhi Kahani Hai, Fail Hui Kahani Nahi

Ye important hai samajhna — ye poora episode ek 'failure' nahi hai, balki science jaisa kaam karna chahiye, waisa hi kaam kiya hai. Ek naya, exciting result mila (Entry 33). Us result ko turant sach nahi maan liya gaya — external review se guzara gaya. Review ne ek genuine, chhupa hua bug pakda (class 11 = snow, clear ground nahi). Bug fix hua, do independent variants ke saath cross-verify kiya gaya. Result ab poori tarah stable, teen independent tareekon se confirmed hai. Aur is poore process mein, ek *doosra* bug bhi mila aur fix hua — comparison script ka apna hardcoded narrative — jo khud is baat ka proof hai ki 'apne khud ke output par bhi andha bharosa mat karo, use bhi verify karo.'

Is PDF mein humne dekha: SCL band Sentinel-2 ke pixels ko category-codes deta hai (2=dark, 4=vegetation, 5=bare soil, 6=water, 11=snow); `make_mask_fn()` ek closure hai jo classes-list se dynamically ek masking-function banata hai, taaki corrected aur strict variants ek hi template se ban sakein; class 11 ko 'clear' maan lena ek genuine bug tha jisne Himalayan summer-sample mein snow ko galti se built-up jaisa dikha diya (Entry 33's false-significant result); external review ne isse pakda, fix hua do variants ke saath (Entry 34); teeno masks (QA60, SCL-corrected, SCL-strict) ab consistently null result dete hain, strong village-level agreement ke saath; comparison script mein khud ek doosra bug tha — hardcoded narrative-text jo naye data ke saath purani kahani keh raha tha — jo dynamic, data-se-compute-hone-wale verdict se fix hua; aur ek coverage-anomaly (200 se 251) honestly open-question ki tarah record ki gayi, guess ko fact banaye bina.

Agla PDF (PDF 10) acquisition ke baaki teen files cover karega — multi-year data, pretreatment baseline, aur secondary outcomes. Jab bhi tayyar ho, bata dena.